

[Necessità di Rappresentare Esecuzione di una Funzione]

$f: \text{INPUT} \rightarrow \text{OUTPUT}$

Mi Serve Strumento per descrivere come Calcolare f

ovvero Vogliamo Rappresentare ALGORITMI

Sequenza finita di Passi discreti che descrivono come Calcolare f

Mi dice la "STRATEGIA" per Calcolare la Funzione

Mi Serve Strumento che Manipoli dati descrivendo ALGORITMI
PROGRAMMA

Linguaggi di Programmazione $\Rightarrow$ Linguaggi formali che permett. di Scrivere il PROGRAMMA

Programmi Sono Oggetti finiti, Bene formati di un PL
 $\downarrow$
Programming Language

Quindi una Representazione ^{finita} di una Sequenza potenzialmente Infinita di Effetti su una Macchina.

Computazione $\infty \Rightarrow$ Sistema Operativo ad Esempio che mi Viene offerta dal ciclo WHILE che dipende da delle

Condizioni per il Quale Terminerà o Continuerà a Ciclare

Linguaggi ce ne Sono molti e sono Turing Calcolabili e quindi EQUIVALENTI che Sono più o Meno Efficienti in determinati Ambiti

SUDDIVISE PER APPLICAZIONI:

- 🌀 Scientifiche $\Rightarrow$ FORTRAN $\Rightarrow$ Tanti Valori dopo la Virgola
- 🌀 Economiche $\Rightarrow$ COBOL $\Rightarrow$ Con Approssimazioni ed è LEGACY CODE perché non si Vuole Cambiare linguaggio ma si Aggiusta
- 🌀 Intelligenza Artificiale $\Rightarrow$ LISP e TUTTI LINGUAGGI FUNZIONALI
- 🌀 Sistemi $\Rightarrow$ C $\Rightarrow$ Basso lvl ed Efficiente
- 🌀 Web $\Rightarrow$ JAVA/PHP/JAVASCRIPT

LIVELLO:

BASSO:

- 🌀 Linguaggio Macchina
 - 🌀 Assembly
- CHE HA PORTATO

ALTO: (mix. tra formalità e linguaggio Naturale)

- 🌀 PARADIGMI
 - IMPERATIVO $\Rightarrow$ lavora Sulla CELLA di memoria come C
 - FUNZIONALE $\Rightarrow$ ML, LISP... si Basa sulla composizi. di funzioni
 - LOGICO $\Rightarrow$ PROLOG...
- PROCEDURALE $\Rightarrow$ è un IMPERATIVO ma SENZA salti o SUB-ROUTINE
- OBJECT ORIENTED $\Rightarrow$ JAVA
- DECLARATIVE $\Rightarrow$ Logico/Funzionale

Significato di Variabile è molto diverso:

- 1/ **IMPERATIVO** $\Rightarrow$ Astrazione della cella, modifichiamo il suo contenuto
- 1/ **FUNZIONALE** $\Rightarrow$ Classica Variabile MATEMATICA, è un PLACEHOLDER dove tutte le occorrenze hanno stesso valore

GENERAZIONALI

- 1/ **HW PROPRIETARI** $\Rightarrow$ Programma per l'architettura (BINARI)
- 1/ **ASSEMBLY** $\Rightarrow$ Prima Astrazione dal BINARIO
- 1/ **ALTO LIVELLO** $\Rightarrow$ Usano linguaggio Naturale $\Rightarrow$ C, BASIC, JAVA
- 1/ **DICHIARATIVI** $\Rightarrow$ SQL
- 1/ **APPL ORIENTED** $\Rightarrow$ Per Rispondere ad Esigenze Specifiche

CARATTERISTICHE:

- 1/ **SEQUENZIALI**
- 1/ **CONCURRENTI/MULTI THREAD** $\Rightarrow$ c'è Sincronizzazione e quindi c'è Comunicazione
- 1/ **Parallelo** $\Rightarrow$ Non servono costrutti per far comunicare
- 1/ **Distribuito** $\Rightarrow$ Su più macchine di calcolo
- 1/ **Object Oriented** $\Rightarrow$
- 1/ **Scripting** $\Rightarrow$

3 ASPETTI IMPORTANTI

- ① LEGGIBILITÀ
 - ② WRITEABILITY
 - ③ AFFIDABILITÀ
- Semplicità, #Limitata di Costrutti, poco OVERLOADING
e poca Moltiplicità
→ modi $\neq$ di dire la stessa cosa
- operatore ha più significati

ORTOGONALITÀ DEL LINGUAGGIO (Linguaggi TIPI)

Astrazione (o.o) ed Espressività

Type Checking, a Tempo Statico di Compilazione Posso Eseguire dei CHECK ed Elimino gli ERRORI di TIPO

Gestione degli ECCEZIONI

ALIASING $\Rightarrow$ Effetto Negativo per Avere Nomini $\neq$ per la Stessa Cella (LINGUAGGI con PUNTATORI)

IMPLEMENTARE UN LINGUAGGIO:

(PL)
↳ Programming Language

Costruire **INFRASTRUTTURA** da Installare sulle Macchine (fisica) per far sì che possa Eseguire il PL.

Bisogna Porre Attenzione a:

/// **ARCHITETTURA DELLA MACCHINA**



Ho dati e Programmi in MEMORIA (che è SEPARATA dalla CPU) ma anche Istruzioni e dati che viaggiano tra memoria e CPU

ESecuzione Macchina Fisica

Program Counter

Repeat forever

fetch Instruction Nella MEM Usando PROGRAM COUNTER

aggiorno Program Counter

decode Instruction

Execute Instruction

Implementare PL = Costruire macchina Astratta per il mio PL

MACCHINA ASTRATTA $\Rightarrow$ Alto Livello

MACCHINA FISICA $\Rightarrow$ Programmi linguaggio Macchina

PL $\Rightarrow$ Linguaggio L con i Suoi Costrutti Sintattici



frase Ben formata Nel PL = Programma

Per Eseguirlo Costruiamo La Macchina Astratta per L

MACCHINA ASTRATTA:

Dato L Linguaggio di Programmazione, M_L è Macchina Abs. di L
è un Insieme di Strutture dati ed Algoritmi che **Permettona di**
Eseguire Programmi Scritti in L.

Quindi ho Max. Velocità ma **ZERO FLESSIBILITÀ**... cambio
Linguaggio (L) e devo Cambiare Macchina (M)

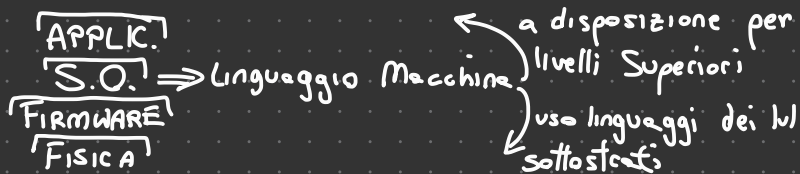
LINGUAGGIO MACCHINA:

Dato M Macchina Abs. il Suo linguaggio L_M è il linguaggio
macchina di M, ed è l'Insieme di Tutte Le Stringhe di Tutte Le
Stringhe Eseguibili (riconoscibili) da M.

TEOREMA DEL PADOVA $\Rightarrow$ Posso Avere ∞ Macchine per l'
Implementazione di **1 Linguaggio**

Così Posso diventare Indipendente dalla Macchina Fisica

ESEMPIO DI LIVELLI:



Implementare un PL $\Rightarrow$ Realizzare M_L

IMPLEMENTAZIONE:

- ① TRADUZIONE IMPLICITA $\Rightarrow$ Interprete
- ② TRADUZIONE ESPLICITA $\Rightarrow$ Compilatore

Esistono anche degli IBRID, ma c'è una PREDOMINANZA di una delle 2 Caratteristiche

① Simula le Operazioni, traducendole via via che le Eseguo in linguaggio Sottostante.

② Realizzo M₁ Traducendo Intermemente il Programma Scritto in L in un Programma Scritto in linguaggio Sottostante
(PENSARE AL PROCESSO DI COMPILAZIONE IN JAVA)

Linguaggi Compilati spesso più EFFICIENTI visto che hanno TRADUZIONE OFFLINE visto che non è a periodo di ESECUZIONE

Non È decidibile sapere se 2 Programmi Sono Equivalenti, è una conseguenza del TH. DI RICE

NOTAZIONE:

$\text{prog}^L \Rightarrow$ Insieme dei Programmi Scritti in L

$D \Rightarrow$ Insieme dei dati

$P^L \Rightarrow$ Programma Specifico Scritto in L , $P^L: D \rightarrow D$

$\text{input} \in D$ $P^L(\text{Input}) = \text{output}$

Esecuzione di P^L su un dato Input fornisce uno specifico OUTPUT

INTERPRETE:

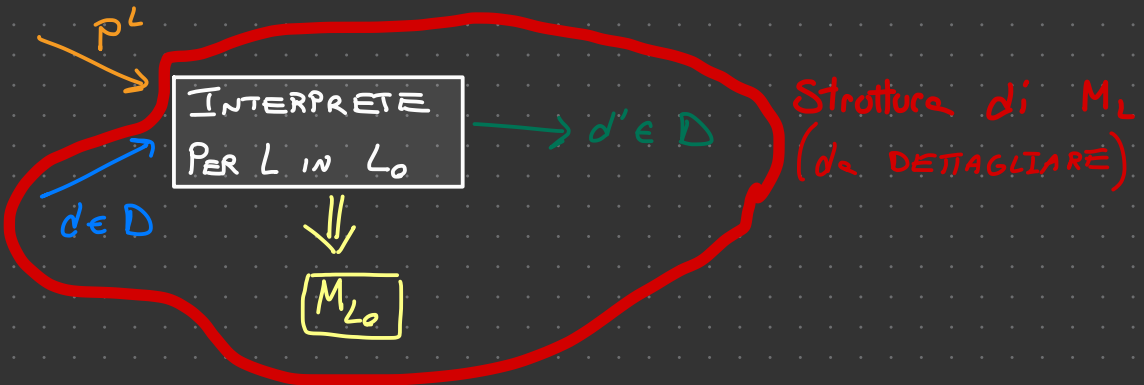
Per il linguaggio L , scritto in L_0 (linguaggio sottostante) è un programma

$$I^{L_0 L}: \text{Prog}^L \times D \rightarrow D$$

INPUT $\leftarrow$ $\rightarrow$ OUTPUT

$\forall \text{input} \in D. \quad I^{L_0 L}(P^L, \text{input}) = \text{OUTPUT} \quad \forall P^L \in \text{Prog}^L$

che è una MACCHINA UNIVERSALE





METODOLOGIE DI PROGETTAZIONE DI PROGRAMMI